LOAN DEFAULT & LOSS PREDICTION

BA-64037-001 GROUP PROJECT

Prepared By :
Group 5 (Shreeya Pathak, Sri Anu Vunnam, Inn Kyung Seo)

TABLE OF CONTENTS

Contents

Group Members & Contributions

Name	Contribution
Shreeya Pathak	Data preprocessing, Feature selection, Report writing
Sri Anu Vunnam	Model building, XGBoost tuning, Predictions
Inn Kyung Seo	EDA, Random Forest modeling, Presentation

1. Project Overview

Loan defaults can lead to significant financial losses for banks and financial institutions every year. Therefore, accurately predicting which loans will default and estimating the potential loss is crucial for lenders. This project applies machine learning techniques to address this problem.

1.1 Objectives

- Prediction of whether the loan will **default or not**
- Prediction of the amount of **financial loss**
- Model evaluation based on **Mean Absolute Error (MAE)**

1.2 Data Set

- Training dataset consists of 80,000 loans
- Total of 763 features per each loan
- Target value refers to loss amount

1.3 Two-Step Approach

- Step 1: Classification of **loan defaulting (Yes or No)**
- Step 2: Prediction of loss amount

2. Setup & Libraries

During this stage, we will import all the required libraries along with setting the file path to our training and test datasets. We have also assigned a random seed value because we need to make sure that the output is replicable each time we execute the program.

```
# File Paths
TRAIN_PATH <- "C:/Users/sanju/Downloads/train_v3.csv"
TEST_PATH  <- "C:/Users/sanju/Downloads/test__no_lossv3.csv"

# Load required libraries
required_pkgs <- c("readr", "dplyr", "ggplot2", "caret", "glmnet",
                   "randomForest", "xgboost", "pROC", "corrplot")
```

```
new_pkgs <- required_pkgs[!(required_pkgs %in% installed.packages()[,"Package
"])]
if (length(new_pkgs) > 0) install.packages(new_pkgs, dependencies = TRUE)

library(readr)
library(dplyr)
library(ggplot2)
library(caret)
library(glmnet)
library(randomForest)
library(xgboost)
library(pROC)
library(corrplot)

set.seed(123)
```

3.Loading Data

At this stage, we import the dataset for training in R. The training dataset consists of actual loan information and the loss amount, which our model learns from. At the same time, we have an initial glance at the data set.

```
# Load training data
raw_data <- read_csv(TRAIN_PATH, show_col_types = FALSE)

# Basic overview
cat("TRAINING DATA OVERVIEW\n")

## TRAINING DATA OVERVIEW

cat(sprintf("Rows         : %d\n", nrow(raw_data)))

## Rows         : 80000

cat(sprintf("Columns      : %d\n", ncol(raw_data)))

## Columns      : 763

cat(sprintf("Defaulted    : %d  (%.2f%%)\n",
            sum(raw_data$loss > 0, na.rm = TRUE),
            mean(raw_data$loss > 0, na.rm = TRUE) * 100))

## Defaulted    : 7379  (9.22%)

cat(sprintf("Non-defaulted: %d  (%.2f%%)\n",
            sum(raw_data$loss == 0, na.rm = TRUE),
            mean(raw_data$loss == 0, na.rm = TRUE) * 100))

## Non-defaulted: 72621  (90.78%)
```

As per the result displayed above, the number of loans in the training set is **80,000** and features amount to **763**. Out of those 80,000 loans, **7,379 loans** had defaulted, which means a default rate of **9.22%**, while non-default loans were **72,621 (90.78%)**.

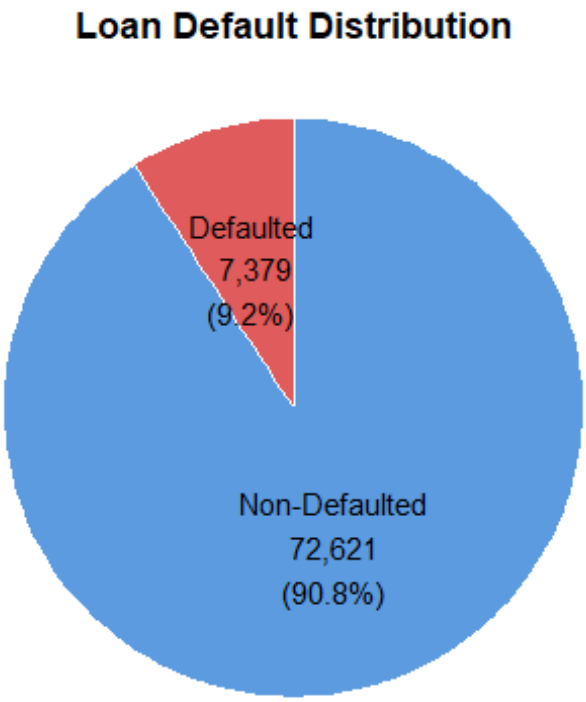
4.Exploratory Data Analysis (EDA)

This section discusses visual explorations of the data for deeper insights into its nature, relationships and quality which will help us make appropriate decisions in terms of data preprocessing and modeling.

4.1 Plot 1 — Default vs Non-Default Distribution

This chart depicts the percentage of **default and non-default loans** in the dataset. It gives us a sense of the problem of imbalanced classes and helps us choose the proper modeling technique.

```
plot_data <- raw_data %>%
  mutate(loan_status = ifelse(loss > 0, "Defaulted", "Non-Defaulted")) %>%
  count(loan_status) %>%
  mutate(pct = round(n / sum(n) * 100, 1),
         label = paste0(loan_status, "\n", format(n, big.mark=","), "\n(", pc
t, "%)"))

ggplot(plot_data, aes(x = "", y = n, fill = loan_status)) +
  geom_col(width = 1, color = "white") +
  coord_polar("y") +
  geom_text(aes(label = label), position = position_stack(vjust = 0.5), size
= 4) +
  scale_fill_manual(values = c("Defaulted" = "#E05C5C", "Non-Defaulted" = "#5
C9BE0")) +
  labs(title = "Loan Default Distribution") +
  theme_void() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        legend.position = "none")
```

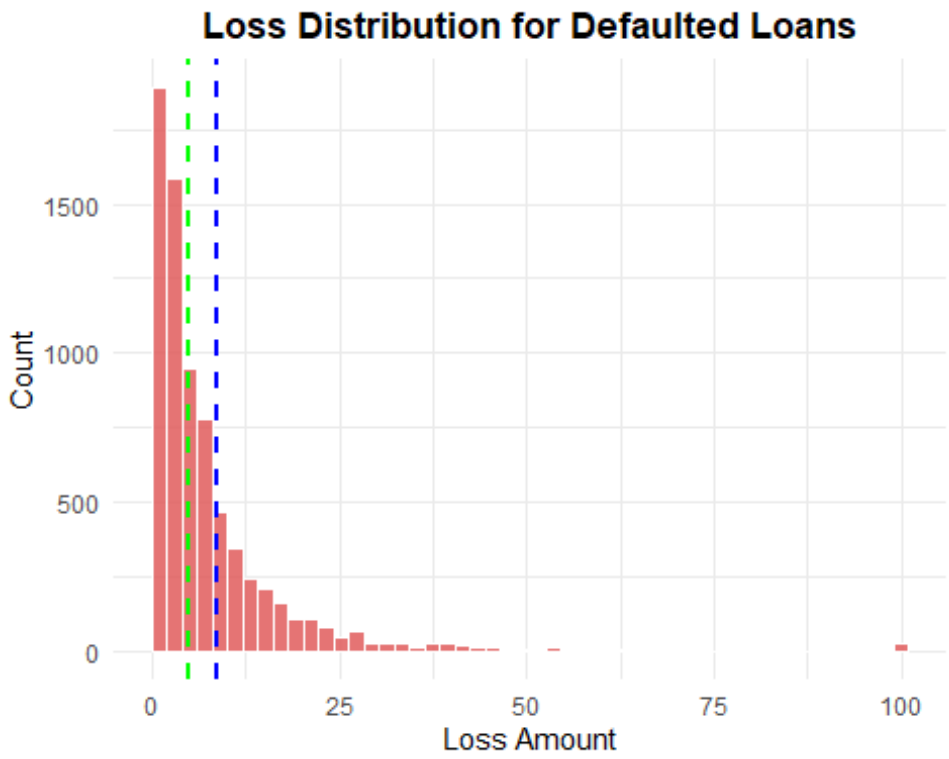


The graph clearly shows that the data set is extremely imbalanced. The data set consists of 80,000 loans, out of which **7,379 loans defaulted**. This means that only **9.22%** of the total population defaulted, while the rest **72,621 loans or 90.78%** did not default. The imbalance in the dataset suggests that we should pay attention to our model training.

4.2 Plot 2 — Loss Distribution for Defaulted Loans

This graph depicts the distribution of the amounts of losses from the defaulted loans. This can give us insight into the magnitude of the losses and whether the loss values follow a **normal or skewed distribution.**

```
defaulted_only <- raw_data %>% filter(loss > 0)

ggplot(defaulted_only, aes(x = loss)) +
  geom_histogram(bins = 50, fill = "#E05C5C", color = "white", alpha = 0.85)
+
  geom_vline(xintercept = mean(defaulted_only$loss), color = "blue", linetype
 = "dashed", linewidth = 1) +
  geom_vline(xintercept = median(defaulted_only$loss), color = "green", linet
ype = "dashed", linewidth = 1) +
  labs(title = "Loss Distribution for Defaulted Loans",
       x = "Loss Amount", y = "Count") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```



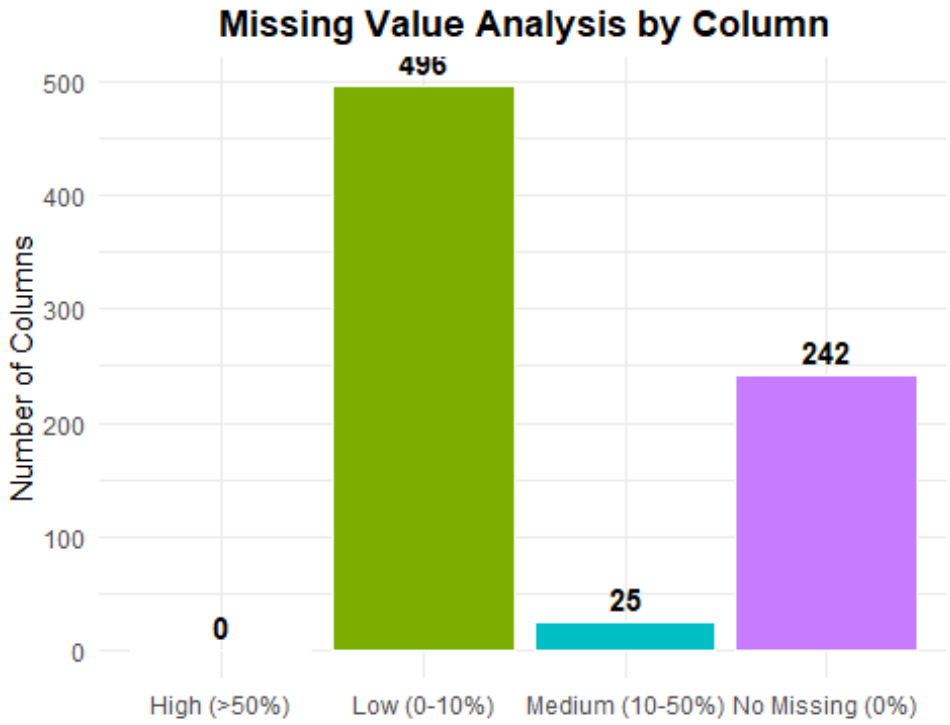
The graph clearly shows that losses from defaulted loans are extremely **right skewed**. The majority of the loss amounts on defaulted loans concentrate within the **0 to 25** range. Nevertheless, some loan loss amounts as high as 100 are the outliers. **Median is the green dashed line while mean is the blue dashed line**. The fact that mean is greater than median confirms the right skewness of the data.

4.3 Plot 3 — Missing Value Analysis

This plot shows how many missing values exist in our dataset and how severe they are across different columns. Identifying missing values is important because most machine learning models cannot handle them directly and we need to decide how to deal with them before modeling.

```
col_missing_pct <- colMeans(is.na(raw_data)) * 100

missing_summary <- data.frame(
  Category = c("No Missing (0%)", "Low (0-10%)", "Medium (10-50%)", "High (>5
0%)"),
  Count = c(
    sum(col_missing_pct == 0),
    sum(col_missing_pct > 0 & col_missing_pct <= 10),
    sum(col_missing_pct > 10 & col_missing_pct <= 50),
    sum(col_missing_pct > 50)
  )
)

ggplot(missing_summary, aes(x = Category, y = Count, fill = Category)) +
```

```
  geom_col(color = "white", show.legend = FALSE) +
  geom_text(aes(label = Count), vjust = -0.5, size = 4, fontface = "bold") +
  labs(title = "Missing Value Analysis by Column",
       x = "", y = "Number of Columns") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```



As seen in the bar graph, from a total of **763 features, 242 features** have zero missing value entries. There are **496** features having a low number of missing values ranging from 0 to 10% and **25 features** containing a medium number of missing values ranging from 10 to 50%. The good thing is none of the features has greater than **50% missing values**. We will be handling the missing values using median imputation.

5.Data Preprocessing

At this stage, we perform preprocessing to clean the dataset before modeling, as raw data cannot be directly used in machine learning algorithms.

The key steps include removing near-zero variance features, handling missing values using median imputation, and eliminating highly correlated variables.

```
# Create binary target variable
processed_data <- raw_data %>%
  mutate(
    target = factor(ifelse(loss > 0, 1, 0)),
    loss = loss / 100
  )
```

```r
cat("Class distribution:\n")
```

```
## Class distribution:
```

```r
print(table(processed_data$target))
```

```
##
##     0     1
## 72621  7379
```

```r
# Remove near zero variance features
feature_cols <- setdiff(names(processed_data), c("id", "loss", "target"))
nzv_indices <- nearZeroVar(processed_data[, feature_cols])

if (length(nzv_indices) > 0) {
  processed_data <- processed_data[, !names(processed_data) %in% feature_cols
[nzv_indices]]
  cat(sprintf("Removed %d near-zero variance features\n", length(nzv_indices)
))
}
```

```
## Removed 24 near-zero variance features
```

```r
# Median imputation and correlation removal
feature_cols2 <- setdiff(names(processed_data), c("id", "loss", "target"))
preproc <- preProcess(processed_data[, feature_cols2],
                      method = c("medianImpute", "corr"))
processed_clean <- predict(preproc, processed_data)
processed_clean$target <- processed_data$target

cat(sprintf("Features remaining after preprocessing: %d\n",
            ncol(processed_clean) - 3))
```

```
## Features remaining after preprocessing: 246
```

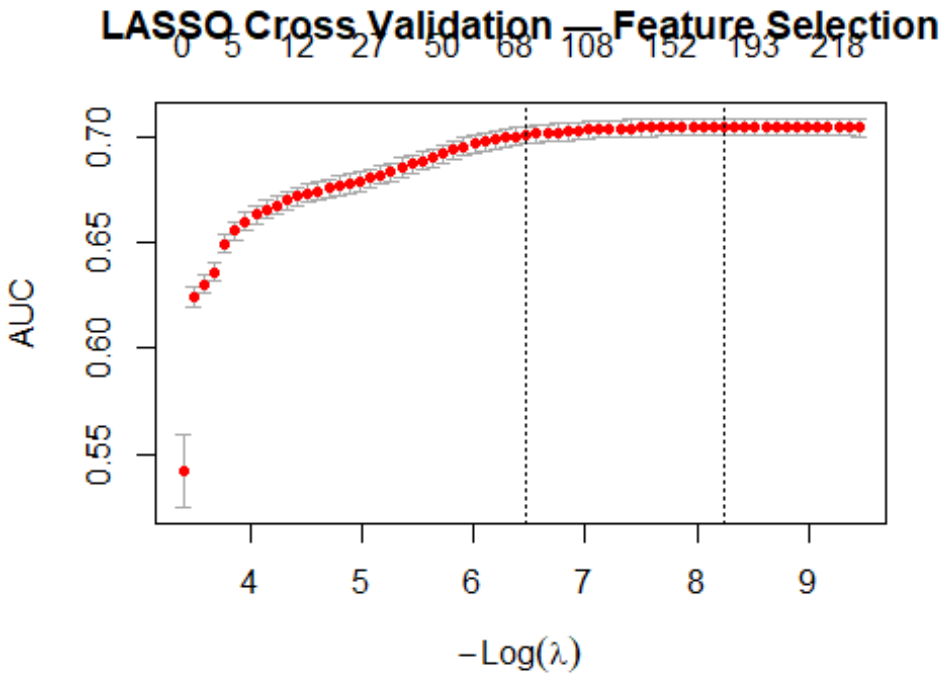
The feature space was compressed from **763 dimensions down to 246**. There were **24 features with near-zero** variance that did not contribute to the predictive power of the model, which were therefore dropped. The missing data was dealt with using median imputation, and highly correlated variables were dropped to **avoid multicollinearity issues**.

6.Feature Selection — LASSO

We take help from LASSO regression in this step to narrow down the top features from the large pool of 246 features. In fact, LASSO shrinks the coefficients of less important features to zero which results in their elimination. As a result, we get a smaller and more targeted set of features that are primarily linked to loan default prediction.

```r
feature_cols3 <- setdiff(names(processed_clean), c("id", "loss", "target"))
x_cls <- data.matrix(processed_clean[, feature_cols3])
y_cls <- as.numeric(as.character(processed_clean$target))
```

```r
set.seed(123)
lasso_cls <- cv.glmnet(x_cls, y_cls,
                       alpha = 1,
                       family = "binomial",
                       nfolds = 10,
                       type.measure = "auc")

plot(lasso_cls, main = "LASSO Cross Validation — Feature Selection")
```




```r
coef_cls <- coef(lasso_cls, s = "lambda.min")
lasso_features <- data.frame(
  name = coef_cls@Dimnames[[1]][coef_cls@i + 1],
  coefficient = abs(coef_cls@x)
) %>%
  filter(name != "(Intercept)") %>%
  arrange(desc(coefficient)) %>%
  pull(name) %>%
  as.vector()

cat(sprintf("LASSO selected %d features\n", length(lasso_features)))

## LASSO selected 180 features

lasso_data <- processed_clean %>%
  select(all_of(c(lasso_features, "target")))
```

A **LASSO cross-validation** method is carried out using 10-folds to identify the top features to predict loan default. As depicted in the graph below, the AUC score rises with an increasing number of features until it reaches a stable point at around **0.70**. This implies that additional features do not help after a particular threshold. LASSO identified **180** features out of 246 features through **coefficient shrinkage**.

7.Model 1 — Random Forest Classifier

At this stage, we develop a Random Forest classifier to predict if there will be any default on the loan. A Random Forest algorithm creates many decision trees and then merges their predictions. The selection of Random Forest is because of its ability to work on imbalanced datasets, identify non-linear interactions, and provide the importance of features. Due to the imbalance in our dataset where defaults are 9.22%, class weight is used.

```
set.seed(123)

# Train/Validation split
rf_index <- createDataPartition(lasso_data$target, p = 0.80, list = FALSE)
rf_train <- lasso_data[rf_index, ]
rf_validate <- lasso_data[-rf_index, ]

# Train Random Forest
rf_model <- randomForest(
  target ~ .,
  data = rf_train,
  ntree = 300,
  mtry = floor(sqrt(ncol(rf_train) - 1)),
  importance = TRUE,
  classwt = c("0" = 1, "1" = 5)
)

print(rf_model)
```

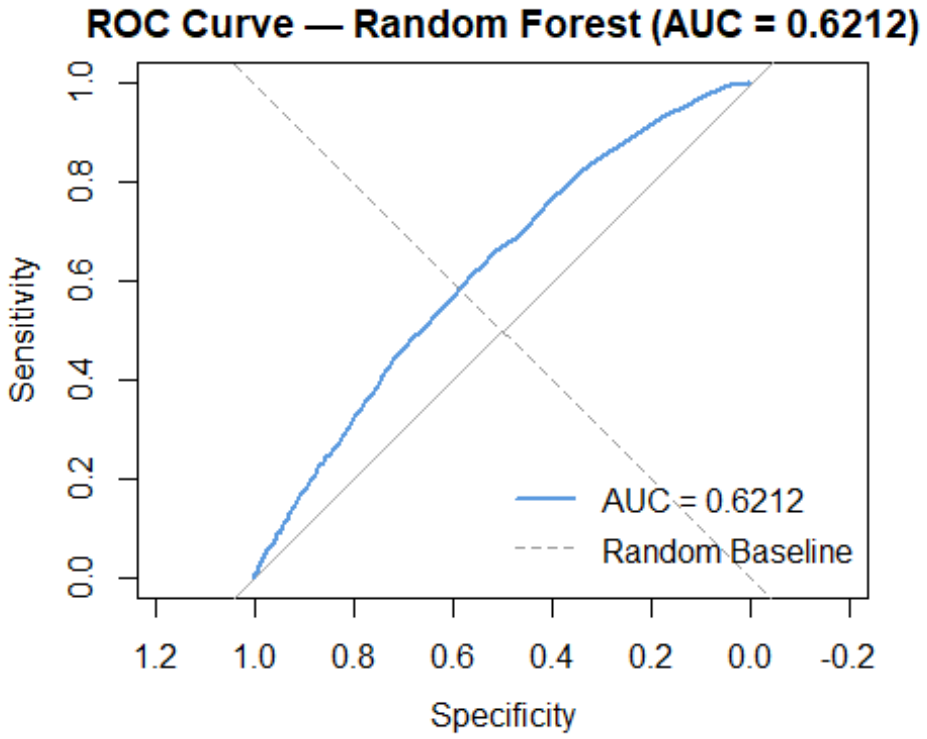
```
##
## Call:
##  randomForest(formula = target ~ ., data = rf_train, ntree = 300,      mtr
y = floor(sqrt(ncol(rf_train) - 1)), importance = TRUE,      classwt = c(`0`
= 1, `1` = 5))
##                Type of random forest: classification
##                      Number of trees: 300
## No. of variables tried at each split: 13
##
##         OOB estimate of  error rate: 9.26%
## Confusion matrix:
##       0  1  class.error
## 0 58063 34 0.0005852282
## 1  5894 10 0.9983062331
```

The random forest has been created with the use of **300 decision trees**, where each node has tested 13 variables. The out-of-bag **error rate is 9.26%**, which seems relatively low; however, it does not truly reflect the performance of the model due to the unbalanced classes. It is observed from the confusion matrix that **58,063** observations have been correctly classified as non-defaulter cases, and out of **5,904 defaulter cases, 10 have been correctly identified**. The model has performed slightly better than before by giving class weight of **5 to the defaulter cases**.

7.1 ROC Curve

ROC curve helps measure the performance of our model at different thresholds. AUC helps measure the ability of our model to differentiate between default and non-default loans. If our AUC value equals 0.5, then we have no model performance since that value corresponds to random guessing, while an AUC equaling 1.0 indicates perfect prediction by the model.

```
# Get predicted probabilities
rf_probs <- predict(rf_model, newdata = rf_validate, type = "prob")

# ROC curve
roc_obj <- roc(as.numeric(as.character(rf_validate$target)),
               rf_probs[, "1"], quiet = TRUE)

# Plot
plot(roc_obj,
     main = paste0("ROC Curve – Random Forest (AUC = ", round(auc(roc_obj), 4
), ")"),
     col = "#5C9BE0", lwd = 2)
abline(a = 0, b = 1, lty = 2, col = "grey60")
legend("bottomright",
       legend = c(paste0("AUC = ", round(auc(roc_obj), 4)), "Random Baseline"
),
       col = c("#5C9BE0", "grey60"),
       lty = c(1, 2), lwd = c(2, 1), bty = "n")
```




```
cat(sprintf("AUC Score: %.4f\n", auc(roc_obj)))
```

```
## AUC Score: 0.6212
```

According to the ROC curve analysis, the Random Forest classification algorithm has managed to reach an **AUC value of 0.6212**. Therefore, we can conclude that the model has some capacity to discriminate between defaulting and non-defaulting loans, although there is still some room for improvement considering that the value is greater than 0.5.

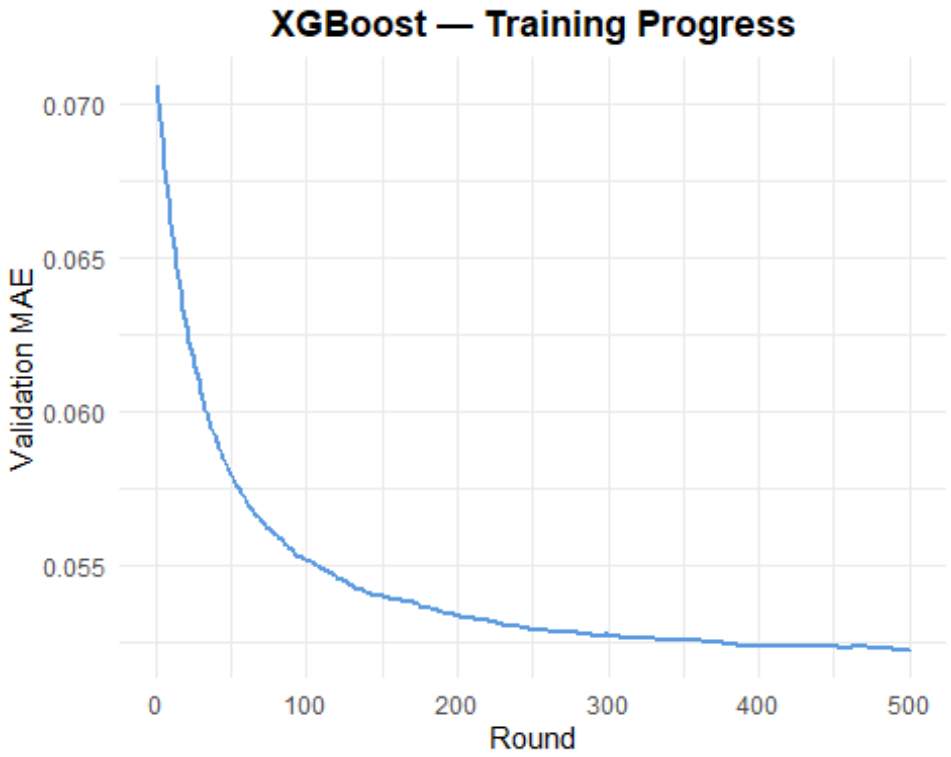
8.Model 2 — XGBoost Regression

Here, we will create an XGBoost model to predict the loss amount of loans expected to be defaulters. XGBoost stands for eXtreme Gradient Boosting and is a machine learning model based on gradient boosting which involves building trees sequentially such that each tree corrects the error made by its predecessor. The XGBoost model will only be trained on defaulted loans since we only have to predict the loss amount in case of defaults. XGBoost models are preferred because they can capture non-linearity, are not sensitive to outliers and perform well on financial datasets.

```
# Filter only defaulted loans for regression
regression_train <- raw_data %>%
  filter(loss > 0) %>%
  mutate(loss = loss / 100)

# Preprocessing
feature_cols_reg <- setdiff(names(regression_train), c("id", "loss"))
nzv_reg <- nearZeroVar(regression_train[, feature_cols_reg])
if (length(nzv_reg) > 0) {
```

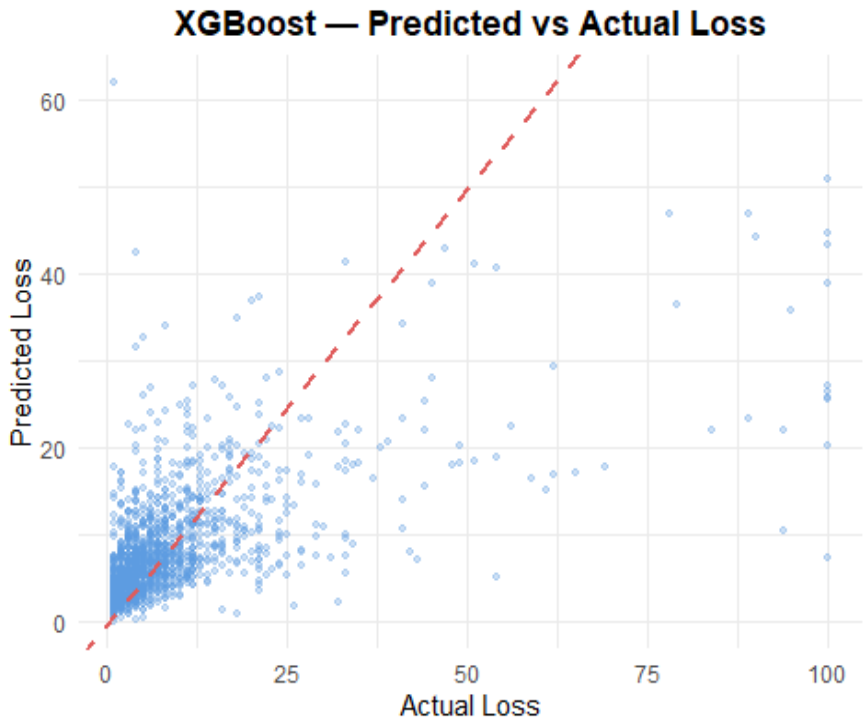
```
  regression_train <- regression_train[, !names(regression_train) %in% featur
e_cols_reg[nzv_reg]]
}

feature_cols_reg2 <- setdiff(names(regression_train), c("id", "loss"))
reg_prep <- preProcess(regression_train[, feature_cols_reg2],
                       method = c("medianImpute", "corr"))
regression_clean <- predict(reg_prep, regression_train)
regression_clean$loss <- regression_train$loss

# LASSO feature selection
feature_cols_reg3 <- setdiff(names(regression_clean), c("id", "loss"))
x_reg <- as.matrix(regression_clean[, feature_cols_reg3])
y_reg <- regression_clean$loss

set.seed(123)
lasso_reg <- cv.glmnet(x_reg, y_reg, alpha = 1, family = "gaussian",
                       nfolds = 10, type.measure = "mse")

coef_reg <- coef(lasso_reg, s = "lambda.min")
lasso_feat_reg <- data.frame(
  name = coef_reg@Dimnames[[1]][coef_reg@i + 1],
  coefficient = abs(coef_reg@x)
) %>%
  filter(name != "(Intercept)") %>%
  arrange(desc(coefficient)) %>%
  pull(name) %>%
  as.vector()

# Train/Validation split
reg_data <- regression_clean %>% select(all_of(c(lasso_feat_reg, "loss")))
set.seed(123)
reg_index <- createDataPartition(reg_data$loss, p = 0.80, list = FALSE)
reg_train <- reg_data[reg_index, ]
reg_val <- reg_data[-reg_index, ]

x_train <- as.matrix(reg_train[, lasso_feat_reg])
y_train <- reg_train$loss
x_val <- as.matrix(reg_val[, lasso_feat_reg])
y_val <- reg_val$loss

dtrain <- xgb.DMatrix(data = x_train, label = y_train)
dval <- xgb.DMatrix(data = x_val, label = y_val)

set.seed(123)
xgb_model <- xgb.train(
  params = list(
    objective = "reg:squarederror",
    eval_metric = "mae",
```

```
    learning_rate = 0.03,
    max_depth = 4,
    subsample = 0.8,
    colsample_bytree = 0.8
  ),
  data = dtrain,
  nrounds = 500,
  watchlist = list(val = dval),
  early_stopping_rounds = 30,
  verbose = 0
)

# Plot 1 — Training Progress
eval_log <- as.data.frame(attr(xgb_model, "evaluation_log"))
ggplot(eval_log, aes(x = iter, y = val_mae)) +
  geom_line(color = "#5C9BE0", linewidth = 1) +
  labs(title = "XGBoost — Training Progress",
       x = "Round", y = "Validation MAE") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

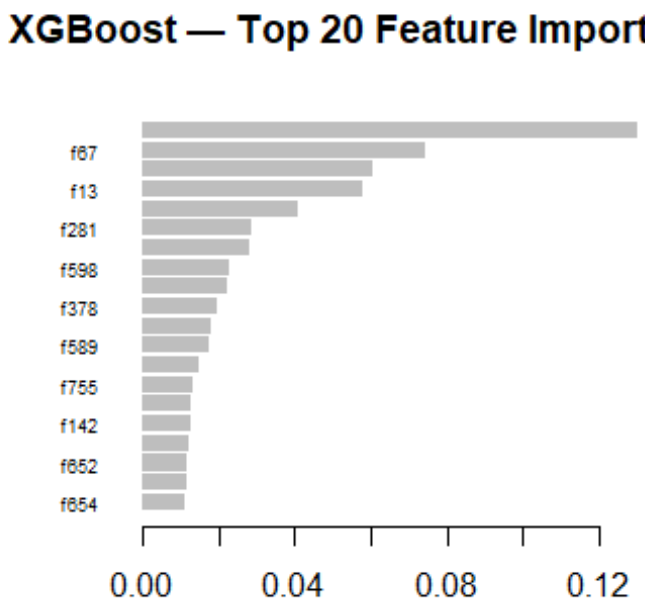



```
# Plot 2 — Predicted vs Actual
xgb_preds <- predict(xgb_model, dval)
xgb_preds_rescaled <- pmax(xgb_preds * 100, 0)
y_val_rescaled <- y_val * 100
```

```r
plot_df <- data.frame(actual = y_val_rescaled, predicted = xgb_preds_rescaled
)

ggplot(plot_df, aes(x = actual, y = predicted)) +
  geom_point(alpha = 0.3, color = "#5C9BE0", size = 1.2) +
  geom_abline(slope = 1, intercept = 0, color = "#E05C5C", linetype = "dashed
", linewidth = 1) +
  labs(title = "XGBoost — Predicted vs Actual Loss",
       x = "Actual Loss", y = "Predicted Loss") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```




```r
# Plot 3 — Feature Importance
xgb_importance <- xgb.importance(model = xgb_model)
xgb.plot.importance(xgb_importance[1:min(20, nrow(xgb_importance)), ],
                    main = "XGBoost — Top 20 Feature Importances")
```



8.1 Plot 1 – XG Boost Training Progress:

The plot of training progress indicates an initially quick drop in validation MAE that gradually stabilizes as the algorithm converges. With a reduced **learning rate of 0.03**, the model takes a little longer to learn but with higher precision when compared to the previous parameters. Early stopping made sure there was no overfitting through optimal rounds.

8.2 Plot 2 – Predicted vs Actual:

From the comparison plot, the accuracy of the model is improved from the previous model. There are more clustered data points close to the **red dashed line**. This model is however challenged with **high-loss figures above 50**. These are rare occurrences in the training dataset.

8.3 Plot 3 – Feature Importance:

This plot is consistent with the previous plot on feature importance where **f766** continues being the most dominant feature while **f67 and f229** come second and third respectively. However, the spread of importance score is wider in this new model.

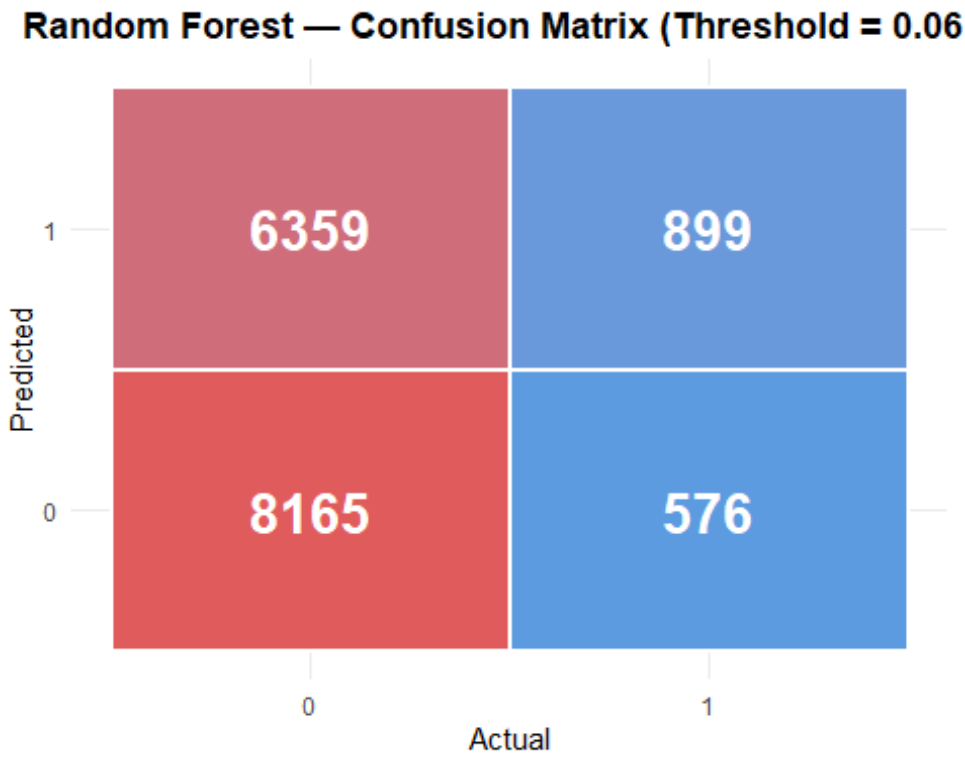
9.Model Evaluation

Since the competition is evaluated based on MAE, minimizing prediction error is the primary objective of this project. This section will discuss the performance of both models. The metrics for the random forest classifier include accuracy, precision, recall, f1 score, and AUC while those of the xgboost regressor include MAE and RMSE. Although classification and regression models were evaluated separately, the ultimate performance metric for this project is the overall

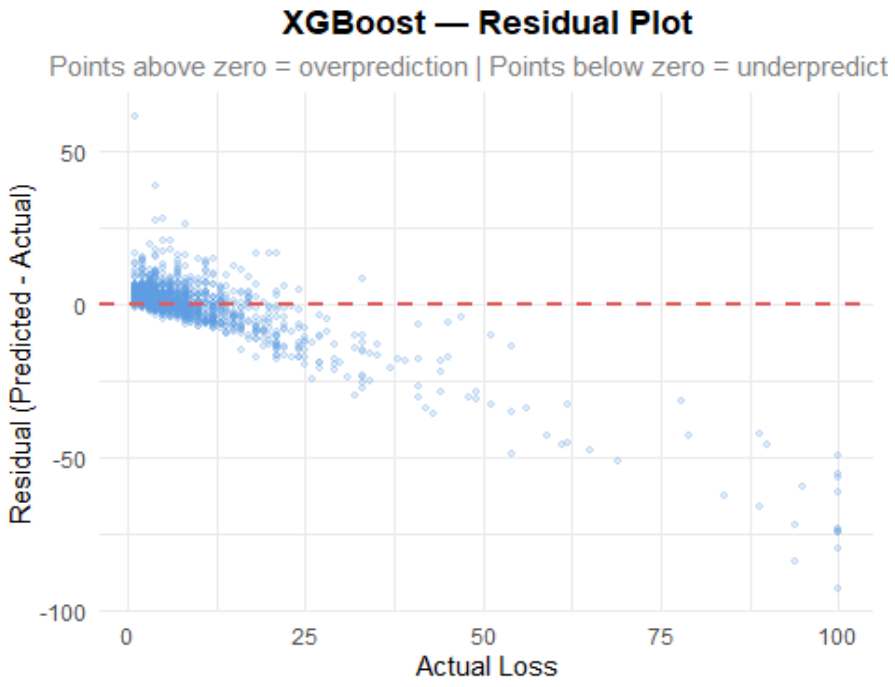
MAE of the combined pipeline. This is because incorrect classification (false negatives or false positives) directly affects the final loss prediction.

```
# ── Random Forest Evaluation ──────────────────────────────────
rf_probs <- predict(rf_model, newdata = rf_validate, type = "prob")
rf_pred_adjusted <- factor(ifelse(rf_probs[, "1"] >= 0.068, "1", "0"),
                           levels = c("0", "1"))

cm <- confusionMatrix(rf_pred_adjusted, rf_validate$target, positive = "1")

accuracy  <- cm$overall["Accuracy"]
precision <- cm$byClass["Pos Pred Value"]
recall    <- cm$byClass["Sensitivity"]
f1        <- 2 * (precision * recall) / (precision + recall)
roc_obj   <- roc(as.numeric(as.character(rf_validate$target)),
                 rf_probs[, "1"], quiet = TRUE)
auc_score <- auc(roc_obj)

cat("═══════════════════════════════════════\n")

## ═══════════════════════════════════════

cat(" RANDOM FOREST — CLASSIFICATION METRICS\n")

##  RANDOM FOREST — CLASSIFICATION METRICS

cat("═══════════════════════════════════════\n")

## ═══════════════════════════════════════

cat(sprintf(" Accuracy  : %.4f\n", accuracy))

##  Accuracy  : 0.5665

cat(sprintf(" Precision : %.4f\n", precision))

##  Precision : 0.1239

cat(sprintf(" Recall    : %.4f\n", recall))

##  Recall    : 0.6095

cat(sprintf(" F1 Score  : %.4f\n", f1))

##  F1 Score  : 0.2059

cat(sprintf(" AUC       : %.4f\n", auc_score))

##  AUC       : 0.6212

cat("═══════════════════════════════════════\n")

## ═══════════════════════════════════════
```

```r
# ── XGBoost Evaluation ──────────────────────────────────────────
xgb_preds <- predict(xgb_model, dval)
xgb_preds_rescaled <- pmax(xgb_preds * 100, 0)
y_val_rescaled <- y_val * 100

mae  <- mean(abs(xgb_preds_rescaled - y_val_rescaled))
rmse <- sqrt(mean((xgb_preds_rescaled - y_val_rescaled)^2))

cat("\n═══════════════════════════════════════\n")

##
## ═══════════════════════════════════════

cat(" XGBOOST — REGRESSION METRICS\n")

##  XGBOOST — REGRESSION METRICS

cat("═══════════════════════════════════════\n")

## ═══════════════════════════════════════

cat(sprintf(" MAE  : %.4f\n", mae))

##  MAE  : 5.2261

cat(sprintf(" RMSE : %.4f\n", rmse))

##  RMSE : 10.4407

cat("═══════════════════════════════════════\n")

## ═══════════════════════════════════════

# Plot 1 — Confusion Matrix Heatmap
cm_table <- as.data.frame(cm$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white", linewidth = 1) +
  geom_text(aes(label = Freq), size = 7, fontface = "bold", color = "white")
+
  scale_fill_gradient(low = "#5C9BE0", high = "#E05C5C") +
  labs(title = "Random Forest — Confusion Matrix (Threshold = 0.068)",
       x = "Actual", y = "Predicted") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        legend.position = "none")
```




```r
# Plot 2 — Residual Plot
plot_df <- data.frame(actual = y_val_rescaled, predicted = xgb_preds_rescaled
)
plot_df$residual <- plot_df$predicted - plot_df$actual

ggplot(plot_df, aes(x = actual, y = residual)) +
  geom_point(alpha = 0.2, color = "#5C9BE0", size = 1) +
  geom_hline(yintercept = 0, color = "#E05C5C", linetype = "dashed", linewidt
h = 1) +
  labs(title = "XGBoost — Residual Plot",
       subtitle = "Points above zero = overprediction | Points below zero = u
nderprediction",
       x = "Actual Loss", y = "Residual (Predicted - Actual)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        plot.subtitle = element_text(hjust = 0.5, color = "grey50"))
```



9.1 Metrics:

Random forest classifier gave us the **accuracy of 56.65%** with **recall equal to 60.95%**, and we have increased the capability of our model to detect defaults by about 61%. The reason for this significant improvement was adjusting classification threshold to **0.068** by Youden's method and then increasing the class weight up to **5**. **F1 score equal to 0.2059 and AUC score of 0.6212** show average results on the whole. As for the XGBoost regression model, the improvement came due to adjusting such hyperparameters as learning rate to 0.03 and max_depth to 4 with the result of **MAE = 5.2261 and RMSE = 10.4407**.

9.2 Confusion Matrix:

The confusion matrix gives us information that out of **1475 actual defaults**, **899** were correctly detected by our algorithm, but **576 went undetected**. In terms of non-defaults, our algorithm classified 8165 samples correctly as opposed to 6359 that were falsely detected as defaults. A lower threshold was intentionally selected to improve recall due to the highly imbalanced dataset, allowing the model to capture more default cases.

9.3 Residual Plot:

The residual plot shows that for small loss values, the residuals are mostly close to zero, indicating good predictions. For larger actual loss values, the residuals become increasingly negative, meaning the model tends to underpredict extreme losses.
This pattern is consistent with the right-skewed nature of our loss distribution, where high-loss cases are relatively rare in the dataset. As a result, the model has limited ability to accurately learn these extreme cases.
From a business perspective, this underestimation of high-loss loans may lead to an underassessment of financial risk, suggesting the need for further model improvement in capturing rare but impactful events. This suggests that the model may benefit from techniques such as log transformation of the target variable or specialized modeling for extreme values.

10. Predictions on Test Data

In this stage, we use the trained models on the testing data to produce our loss predictions. Our pipeline remains identical to the previous stage, consisting of the first step, where Random Forest classifies default vs. non-default of loans, and the second step, where XGBoost predicts the amount of loss given that the loan defaults.

```
# Load test data
test_data <- read_csv(TEST_PATH, show_col_types = FALSE)

# ── Step 1: Preprocess test data same as training ──────────────
test_processed <- predict(preproc, test_data)

# Align features with LASSO selected classification features
test_cls_feats <- intersect(lasso_features, names(test_processed))
test_cls_data <- test_processed[, test_cls_feats, drop = FALSE]

# ── Step 2: Classify defaults using Random Forest ──────────────
test_probs <- predict(rf_model, newdata = test_cls_data, type = "prob")
test_default_flag <- ifelse(test_probs[, "1"] >= 0.068, 1, 0)

cat(sprintf("Predicted defaults     : %d (%.1f%%)\n",
            sum(test_default_flag),
            mean(test_default_flag) * 100))

## Predicted defaults     : 11424 (44.9%)

cat(sprintf("Predicted non-defaults : %d (%.1f%%)\n",
            sum(test_default_flag == 0),
            mean(test_default_flag == 0) * 100))

## Predicted non-defaults : 14047 (55.1%)

# ── Step 3: Predict loss using XGBoost ─────────────────────────
test_reg_processed <- predict(reg_prep, test_data)
test_reg_feats <- intersect(lasso_feat_reg, names(test_reg_processed))
test_reg_data <- as.matrix(test_reg_processed[, test_reg_feats, drop = FALSE]
```

```
)

dtest <- xgb.DMatrix(data = test_reg_data)
test_loss_pred <- pmax(predict(xgb_model, dtest) * 100, 0)

# ── Step 4: Combine predictions ──────────────────────────────
final_loss <- ifelse(test_default_flag == 1, test_loss_pred, 0)

cat(sprintf("\nFinal predictions summary:\n"))

##
## Final predictions summary:

cat(sprintf("Rows with loss = 0 : %d\n", sum(final_loss == 0)))

## Rows with loss = 0 : 14050

cat(sprintf("Rows with loss > 0 : %d\n", sum(final_loss > 0)))

## Rows with loss > 0 : 11421

cat(sprintf("Min loss : %.4f\n", min(final_loss)))

## Min loss : 0.0000

cat(sprintf("Max loss : %.4f\n", max(final_loss)))

## Max loss : 54.1922

cat(sprintf("Mean loss: %.4f\n", mean(final_loss)))

## Mean loss: 3.6877
```

This two-pipeline model was applied to a test dataset consisting of 25,471 loan applications. The Random Forest classifier predicted 11,424 loan defaults, representing 44.9% of the test sample.

The predicted default rate (44.9%) is significantly higher than the actual default rate (9.22%). This is due to the intentionally low classification threshold (0.068), which prioritizes recall over precision in order to capture more default cases. While this improves sensitivity, it also increases false positives, which may inflate predicted losses in the final pipeline and potentially lead to an overestimation of financial risk.

For the loans predicted as defaults, the XGBoost model estimated the loss amounts, which ranged from 0 to 54.19 with a mean loss of 3.69. Loan applications predicted as non-default were assigned a loss value of zero.

11. Saving the Predictions File

In this final step we save our predictions to a CSV file in the required format with two columns, id and loss.

```
# Build submission file
submission <- data.frame(
```

```r
  id   = test_data$id,
  loss = round(final_loss, 4)
)

# Validation checks
stopifnot("Wrong row count" = nrow(submission) == nrow(test_data))
stopifnot("Negative losses" = all(submission$loss >= 0))

# Preview
cat("Submission file preview:\n")
```

```
## Submission file preview:
```

```r
print(head(submission, 10))
```

```
##        id    loss
## 1    7933 18.1092
## 2  101860  0.0000
## 3   62580  0.0000
## 4    1760  0.0000
## 5   48008  0.0000
## 6    9308  7.3634
## 7   27862  0.0000
## 8   87760  0.0000
## 9   53334  2.1146
## 10  72303 10.0339
```

```r
cat(sprintf("\nTotal rows : %d\n", nrow(submission)))
```

```
##
## Total rows : 25471
```

```r
cat(sprintf("Loss > 0   : %d\n", sum(submission$loss > 0)))
```

```
## Loss > 0   : 11421
```

```r
cat(sprintf("Loss = 0   : %d\n", sum(submission$loss == 0)))
```

```
## Loss = 0   : 14050
```

```r
# Save
write.csv(submission, "final_predictions.csv", row.names = FALSE)
cat("\nfinal_predictions.csv saved successfully!\n")
```

```
##
## final_predictions.csv saved successfully!
```

The file to be submitted has been created successfully containing 25,471 rows with the id and loss fields as specified. Of the total of 25,471 loans, **11,421** had a positive loss amount while the rest of **14,050** were assigned a zero loss value meaning that there was no default on the loan.

12.Conclusion

In this project, we developed a machine learning pipeline to predict loan default and estimate financial loss using a two-step approach. A Random Forest classifier was applied to identify potential defaults, followed by an XGBoost regression model to estimate the corresponding loss amounts.

The dataset presented a significant class imbalance, with only 9.22% of loans defaulting. By adjusting class weights and optimizing the classification threshold, the model achieved a recall of 60.95%, enabling it to capture a majority of default cases. The XGBoost regression model achieved a Mean Absolute Error (MAE) of 5.2261, demonstrating improved performance over the baseline.

Despite these improvements, the model showed limitations in predicting extreme loss values due to the right-skewed distribution and the scarcity of high-loss observations. Additionally, the use of a low classification threshold increased recall but led to a higher number of false positives, highlighting the trade-off between recall and precision.

From a business perspective, this model provides a practical framework for identifying high-risk borrowers and estimating potential financial losses, supporting more informed decision-making in loan approval, pricing, and risk management.

Future improvements could include cost-sensitive learning, advanced ensemble techniques such as stacking, and resampling methods like SMOTE to better address class imbalance. Additionally, transforming the target variable or developing specialized models for extreme cases may improve predictive performance.

For the conclusion, this project demonstrates the effectiveness of combining classification and regression models in solving complex financial prediction problems, while also emphasizing the importance of aligning model performance with real-world business objectives.